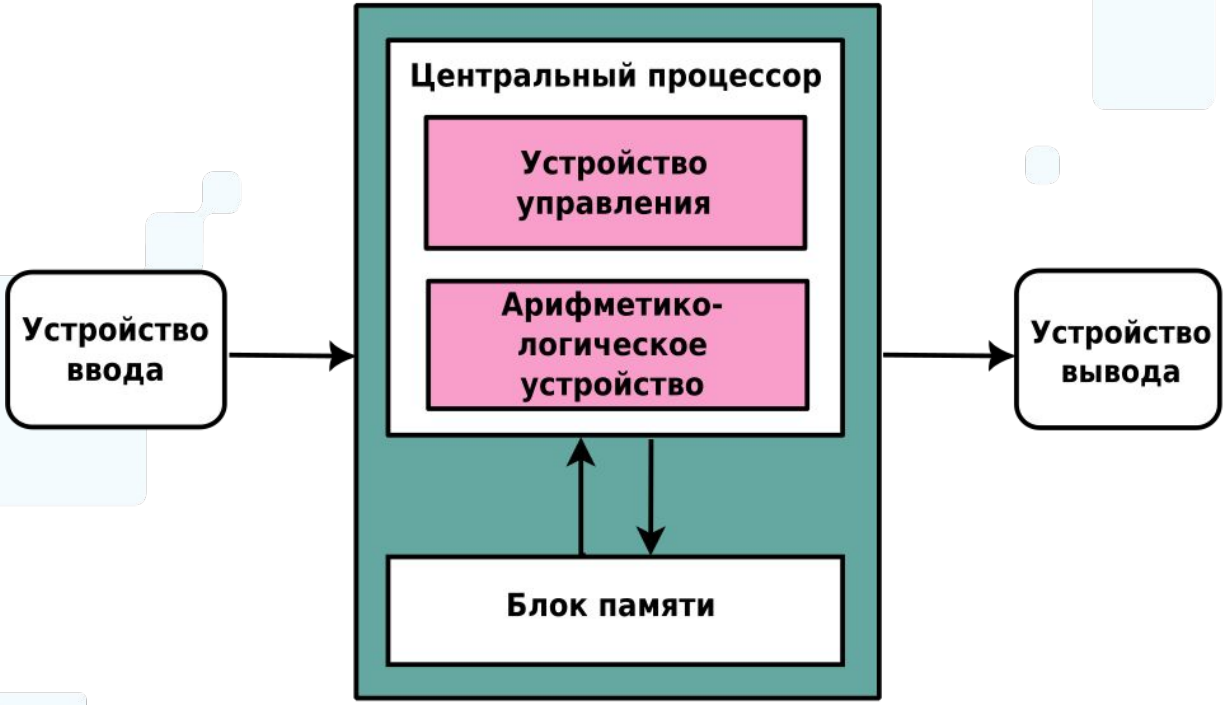
FAST FORWARD

АКОС

Лекция 2
X86_64

Архитектура Фон-Неймана

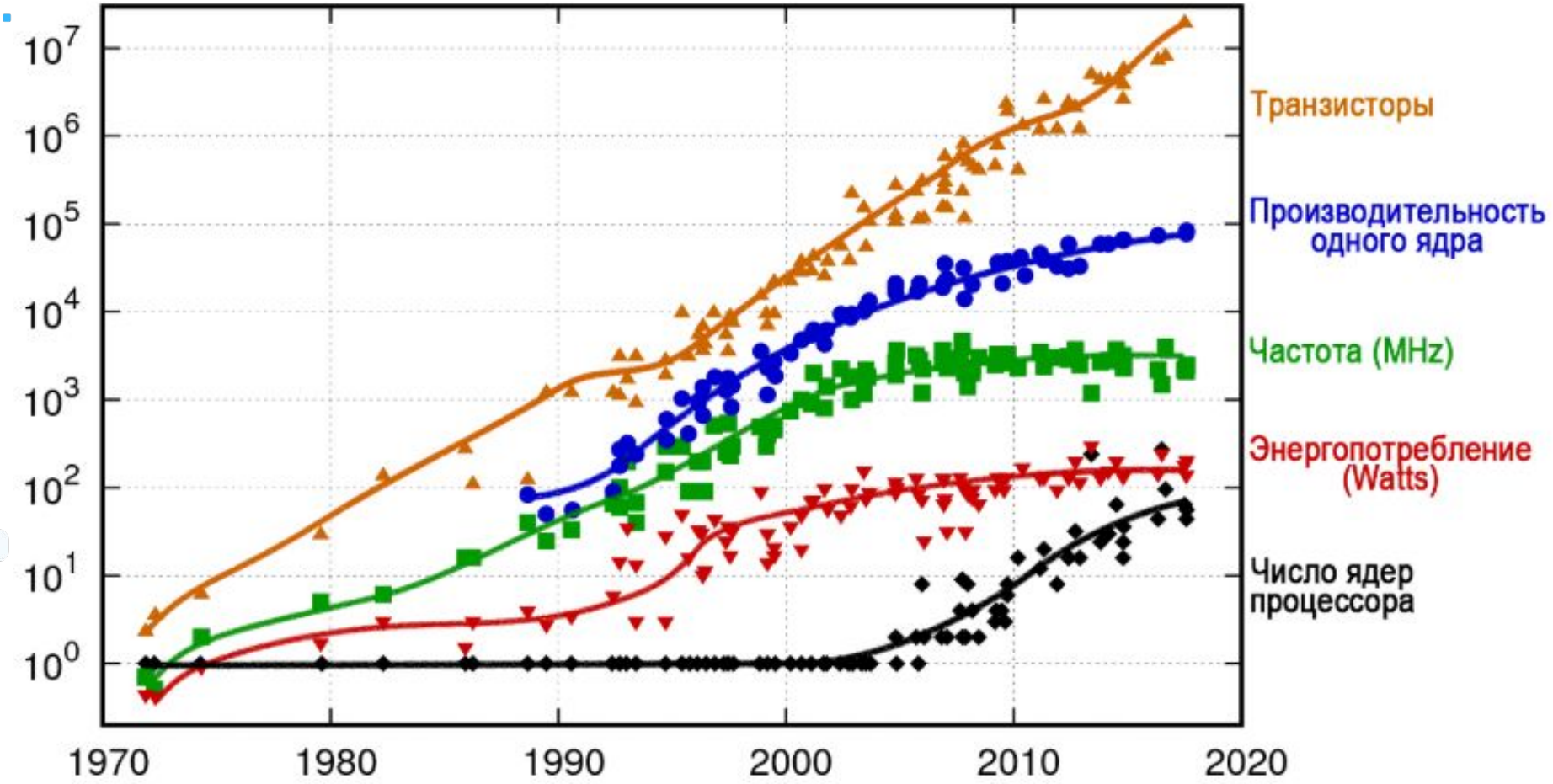


FAST FORWARD

Современный процессор

- Работает быстро
- Работает энергоэффективно
- Работает многопоточно
- Имеет обратную совместимость

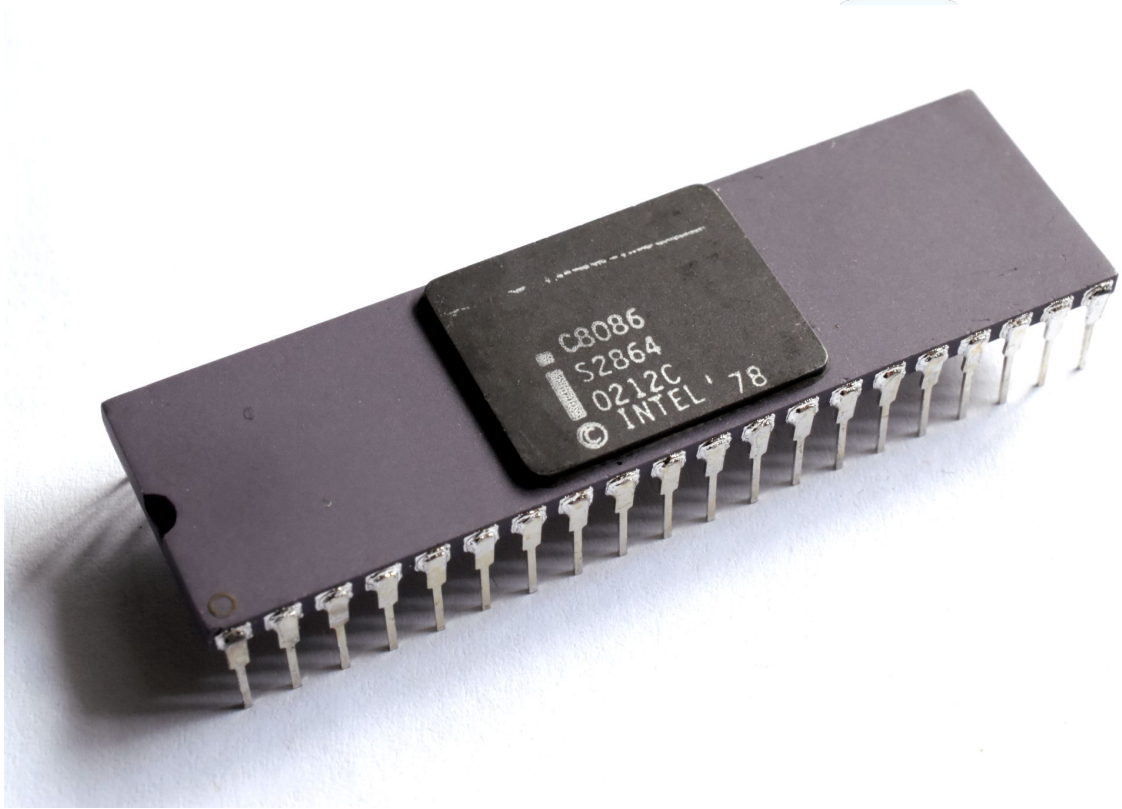
РОСТ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ ПО ГОДАМ



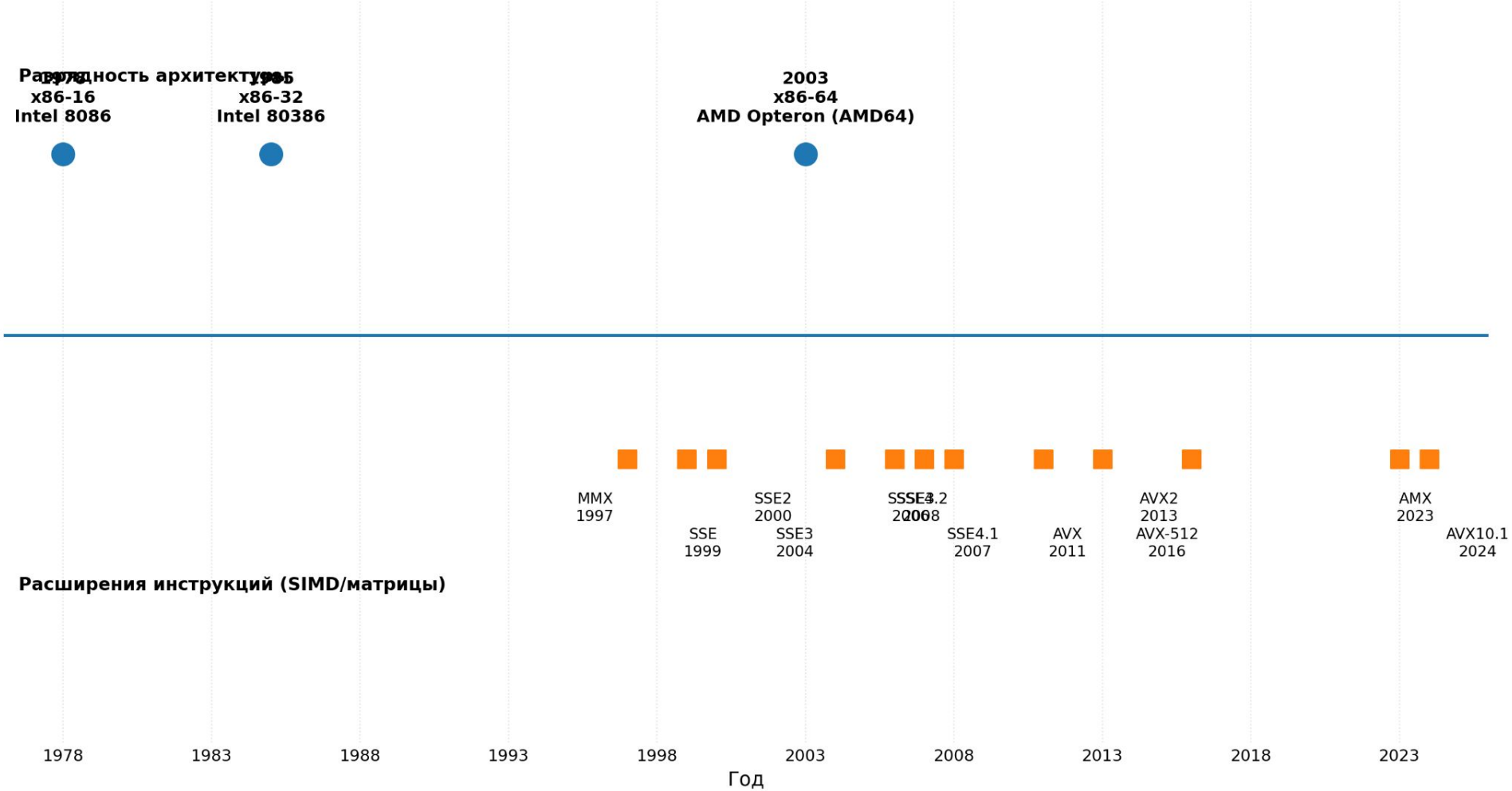
FAST
FORWARD

Intel 8086

- 1978
- 16-bit

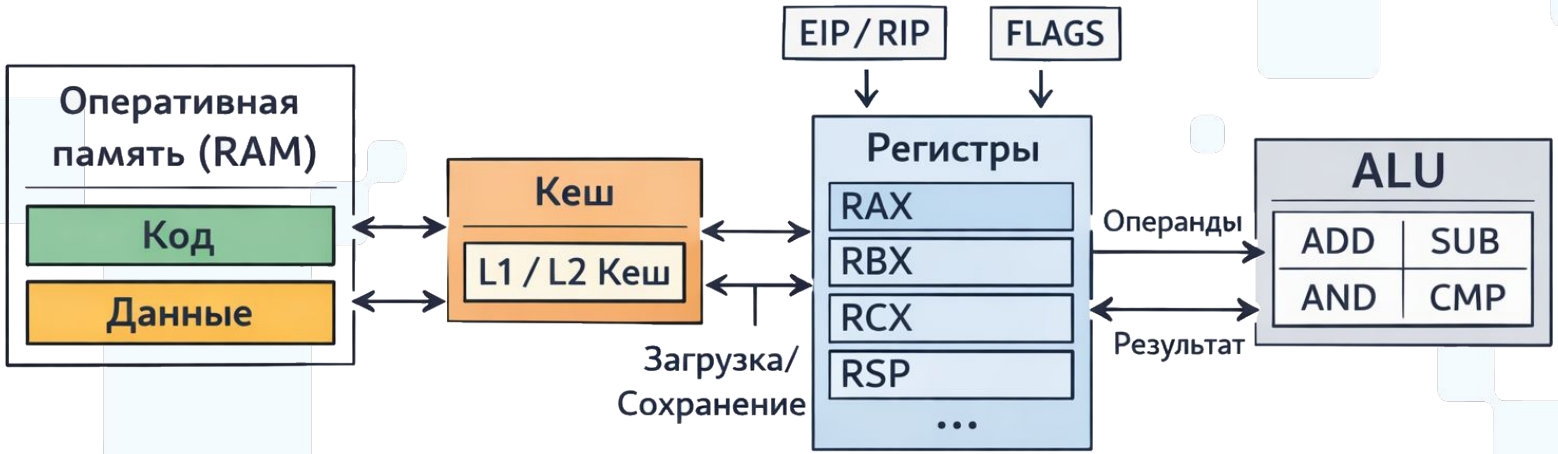


Эволюция x86: разрядность и ключевые расширения ISA



FAST
FORWARD

X86



FAST FORWARD

Целые неотрицательные числа

Num	4bit	64bit
15 = 1111_2	15 = 1111_2	15 = 0…01111_2
0 = 0_2	0 = 0000_2	0 = 0…00000_2
5 = 101_2	5 = 0101_2	5 = 0…00101_2

HEX

0000 0	1000 8
0001 1	1001 9
0010 2	1010 a
0011 3	1011 b
0100 4	1100 c
0101 5	1101 d
0110 6	1110 e
0111 7	1111 f

0xA8 = 10101000_2 = 168

0x16 = 00010110_2 = 22

Знаковые числа 4bit

0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7

1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

Инструкции

39 c8 74 07 77 01 91 29 c8 eb f5 …

Инструкции

```
43210: 39 c8    cmp eax, ecx    // сравнить регистры 0 и 1
43212: 74 07    je 0x4321b      // если числа равны, прибавить к IP число 0x07 = 7
43214: 77 01    ja 0x43217      // если первое число больше, прибавить к IP число 0x01 = 1
43216: 91       xchg eax, ecx   // поменять местами регистры 0 и 1
43217: 29 c8    sub eax, ecx    // вычесть регистр 1 из регистра 0
43219: eb f5    jmp 0x43210     // прибавить к IP число 0xf5 = −9
4321b: ...
```

FAST FORWARD

Регистры

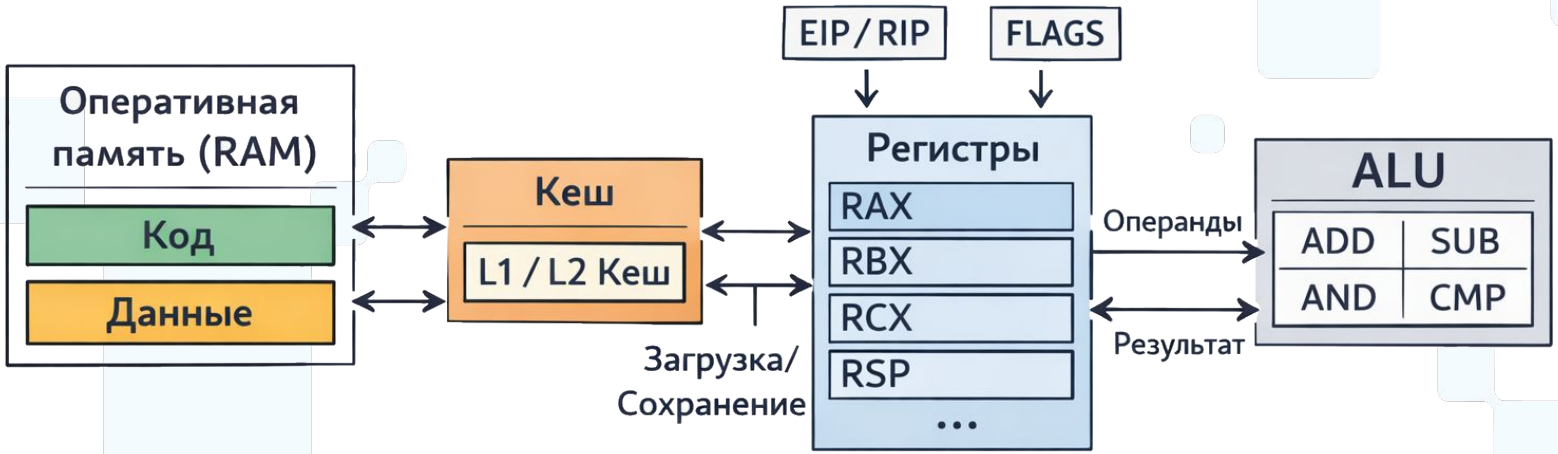
- 64bit
- Не адресуемые

- Общего назначения 16шт
- rip
- rflags

Регистр	Младшие 32 бита	Младшие 16 бит	Два младших байта
rax	eax	ax	ah, al
rbx	ebx	bx	bh, bl
rcx	ecx	cx	ch, cl
rdx	edx	dx	dh, dl
rsi	esi	si	—, sil
rdi	edi	di	—, dil
rsp	esp	sp	—, spl
rbp	ebp	bp	—, bpl
r8	r8d	r8w	—, r8b
...	...	...	...
r15	r15d	r15w	—, r15b

FAST
FORWARD

X86



MOV

```
mov DST, SRC     // копировать SRC в DST

mov ebx, eax    // скопировать биты eax в ebx
                // и выставить старшие 32 бита rbx в 0;
                // старое значение rbx теряется
mov ax, bx
mov ah, bl
```

MOV

```
mov rcx, 42       // положить в rcx битовое представление числа 42

mov rdx, 0x80     // шестнадцатеричная запись операнда

mov eax, -1       // установить все биты eax в 1 (а старшие биты rax в 0)
```

ADD / SUB / INC / DEC

```
mov rax, 0x1234

add  rax, 4          // rax == 0x1238
sub  rax, 56         // rax == 0x1200
inc  rax             // rax == 0x1201
```

FAST FORWARD

Прочие арифметические

```
neg DST          // DST = -DST
not DST          // DST = ~DST
and DST, SRC     // DST &= SRC
or  DST, SRC     // DST |= SRC
xor DST, SRC     // DST ^= SRC
```

А беззнаковые?

$$-8 + 1 = -7$$

$$1000_2 + 1_2 = 1001_2$$

$$8 + 1 = 9$$

Переполнение

$1100_2 + 0111_2 = 0011_2 \quad = 3$

$12 + \quad 7 = 19 \bmod 16 = 3$

FAST FORWARD

Сдвиг

```
mov ax, 0x1234
shr ax, 4          // ax == 0x0123
shl ax, 4          // ax == 0x1230
```

Логический

```
mov ax, 0xfff0     // ax == -16
sar ax, 4          // ax == 0xffff == -1
sal ax, 5          // ax == 0xfff0 == -16
```

Арифметический

FAST FORWARD

Библиотека simpleio

```
call writei32   // напечатать на экране значение edi
                // как знаковое десятичное число
call writei64   // вывести rdi
call readi32    // ввести с клавиатуры число и сохранить в eax
call readi64    // ввести с клавиатуры число и сохранить в rax
call finish     // завершить исполнение программы
```

!!! сохраняет только значения регистров rbx, rsp, rbp, r12–r15 !!!

Пример

```
        .intel_syntax noprefix
        .global main
main:
        call readi64      // считали первое число
        mov r12, rax      // сохранили его в r12
        call readi64      // считали второе число
        add r12, rax      // сложили первое и второе
        mov rdi, r12
        call writei64     // вывели результат
        call finish       // завершили программу
```

FAST FORWARD

Запуск

```
$ gcc -g sum.S simpleio_x86_64.S -o sum
$ ./sum
```

Директивы

```
.intel_syntax noprefix
.global main                // экспортировать символ main
main:
mov eax, 42                 // код в синтаксисе Intel
.att_syntax noprefix
movl $42, eax               // noprefix — без символа %
                            // перед операндами-регистрами
.intel_syntax noprefix
.set answer, 4 * 10         // константа
mov eax, answer + 2
```

FAST FORWARD

Директивы

```
nop                 // “ничего не делать”
.byte 0x90          // байт
.skip 5, 0x90       // пропустить

.word 1, 2, 3       // или .short 1, 2, 3 - 16 bit
.int 4, 5, 6        // 32 bit
.quad 7, 8, 9       // 64 bit
```

FAST FORWARD

Метки и циклы

```
.intel_syntax noprefix
.global main
main:
    xor eax, eax
    xor ebx, ebx
loop:
    inc eax
    inc ebx
    jmp loop
```

Флаги

- Биты регистра rflags
- Имеют “смысл”
- Меняются арифметическими инструкциями
- Не меняются некоторыми инструкциями (например mov)
- stc - установить CF
- clc - сбросить CF
- setc al - установить al в 0/1 в зависимости от флага CF

FAST FORWARD

Флаги

ZF - нулевой результат

SF - отрицательный результат

CF - перенос бита

OF - переполнение знакового результата

PF - четность

Переходы

jmp label - безусловный

jF label - если выставлен флаг F

jnF label - если не выставлен флаг F

FAST FORWARD

Пример

```
    .intel_syntax noprefix
    .global main
main:
    mov eax, 6
loop:
    dec eax
    jnz loop
    call finish
```

Иные комбинации

jg	label	- переход, если больше для знаковых чисел	(ZF=0 and SF=OF)
jge	label	- переход, если >= для знаковых чисел	(SF=OF)
jl	label	- переход, если < для знаковых чисел	(SF≠ OF)
jle	label	- переход, если <= для знаковых чисел	(ZF=1 or SF≠ OF)
ja	label	- переход, если > для беззнаковых чисел	(CF=0 and ZF=0)
jae	label	- переход, если >= (беззнаковый)	(CF=0)
jb	label	- переход, если < (беззнаковый)	(CF=1)
jbe	label	- переход, если <= (беззнаковый)	(CF=1 or ZF=1)

FAST FORWARD

Пример

```
.intel_syntax noprefix
.global main
main:
    xor eax, eax
loop:
    inc eax
    mov ebx, eax
    sub ebx, 10
    jl loop
    call finish
```

FAST FORWARD

Особенности

cmp dst, src - недеструктивная версия sub

test dst, src - недеструктивная версия and

inc/dec - не меняют CF в отличие от add 1

cmovz - условная инструкция mov

adc/sbb - сложение/вычитание с учетом CF

FAST FORWARD

АКОС

QCourse.ru 2026